



EAGLE: 推测抽样需要重新思考问题不确定性

李玉慧[♣] 魏方云[‡] 张超[♣] 张红阳^{♣†}

♣ 北京大学 ‡ 微软研究院 ♣ 滑铁卢大学 † 向量研究所

hongyang.zhang@uwaterloo.ca

<https://github.com/SafeAILab/EAGLE>

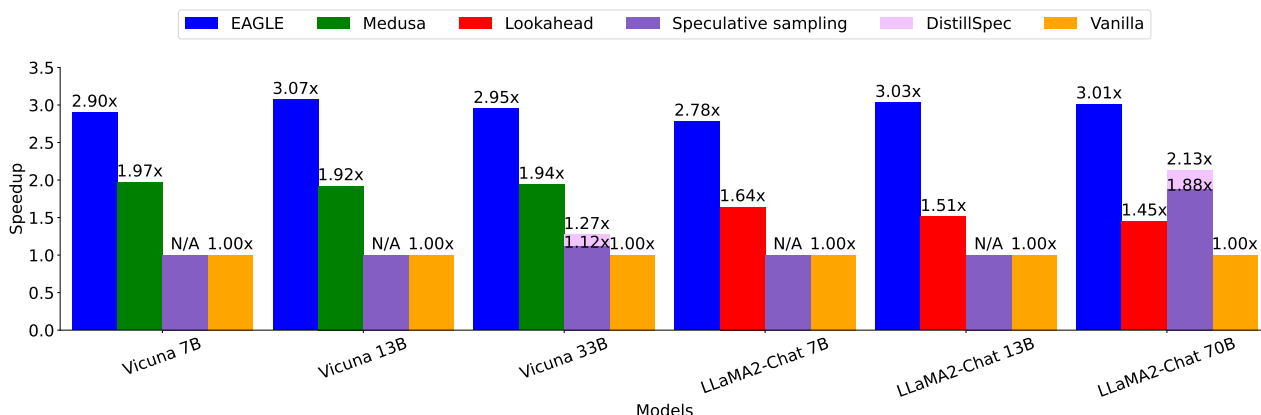


Figure 1: 在贪婪（温度 = 0）设置下，Vicuna 和 LLaMA2-Chat 推理延迟在 MT 平台上的加速比。Medusa 和 Lookahead 的加速比复制自其原始技术报告。通过推测采样，缺乏合适的选秀模型来加速 7B 模型。采用 7B 模型作为 13B 模型的草稿模型会由于 7B 模型的高开销而导致速度缓慢，从而使其效率低于普通自回归解码。这些场景被标记为 N/A。在本文中，我们仅与基于推测采样的方法进行比较，这些方法不需要微调主干模型，确保输出文本分布保持恒定。

Abstract

自回归解码使得大型语言模型 (LLM) 的推理非常耗时。在本文中，我们重新考虑推测性抽样并得出两个关键观察结果。首先，特征（第二层到顶层）级别的自回归比令牌级别更直接。其次，特征（第二层到顶层）级别的自回归固有的不确定性限制了其性能。基于这些见解，我们引入了 EAGLE（提高语言模型效率的外推算法），这是一个简单但高效的推测采样框架。通过合并一个时间步长的令牌序列，EAGLE 有效地解决了不确定性，以最小的开销实现精确的第二层到顶层特征预测。我们对 EAGLE 进行了全面评估，包括 Vicuna 和 LLaMA2-Chat 系列的所有模型、MoE 模型 Mixtral 8x7B Instruct 以及对话、代码生成、数学推理和指令跟踪等任务。对于 LLaMA2-Chat 70B，EAGLE 实现了 2.7 倍-3.5 倍的延迟加速比，吞吐量加倍，同时保持了生成文本的分布。

1. 介绍

自回归解码是大型语言模型 (LLM) 的事实上的标准，它按顺序生成令牌，导致生成缓慢且成本高昂。基于投机抽样 (Leviathan et al., 2023; Chen et al., 2023a) 的方法通过将流程分为低成本草案阶段和针对起草令牌的并行化验证阶段来解决此问题，从而允许在单个 LLM 通行证中验证多个令牌。这些方法通过每次生成多个令牌来加速生成。更重要的是，验证阶段确保文本分布与原始 LLM 的解码结果精确对齐，保持生成内容的完整性。

应用推测性抽样取决于找到一个反映原始 LLM 功能但延迟时间较短的草稿模型，通常涉及同一 LLM 系列的较低参数版本。例如，在包含 7B、13B 和 70B 参数模型的 LLaMA2 (Touvron et al., 2023) 系列中，使用 7B 模型作为 70B 模型的草稿模型是有效的，而为最小的 7B 变体找到合适的草稿模型则很棘手。另一种方法是使用 TinyLLaMA (Zhang et al., 2024)，但由于 LLaMA2-Chat 和 TinyLLaMA-Chat 之间的指令

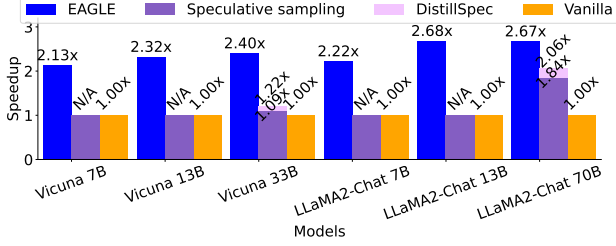


Figure 2: MT 平台上非贪婪（温度 =1）设置的加速比。Lookahead 仅限于贪婪解码，Medusa 的非贪婪生成并不能保证无损性能。因此，EAGLE 不与这些方法进行比较。

模板不一致，因此对于指令调整模型来说不可行。尽管 7B 模型具有作为草稿模型的潜力，但其高开销削弱了加速增益。由于成本高昂，专门为推测性抽样训练新的、大小合适的草图模型也不是理想的解决方案：TinyLLaMA 在 3,000B 令牌上进行训练，而 EAGLE 在 2-4B 令牌上进行训练。

增强投机采样加速的关键在于减少时间开销，提高原 LLM 的稿件接受率 (Chen et al., 2023b; Xia et al., 2023; Santilli et al., 2023)。许多方法都致力于减少起草阶段的开销。Lookahead (Fu et al., 2023) 采用 n-gram 和 Jacobi 迭代，而 Medusa (Cai et al., 2023) 使用一组 MLP，根据原始 LLM 的第二层到顶层特征来预测标记。这些策略显著减少了生成草稿的延迟，从而提高了加速速度。然而，它们的有效性受到所生成草稿的较低准确度的限制，Medusa 的准确度约为 0.6，而 Lookahead 的准确度甚至更低。相比之下，我们的方法的准确度约为 0.8。

为了克服这些限制，我们引入了 EAGLE（提高语言模型效率的外推算法），这是一种有效的推测采样方法，基于以下两个观察结果。

首先，特征级别的自回归比标记级别的自回归更简单。在本文中，“特征”指的是原始 LLM 的第二层特征，位于 LM 头之前。与自然语言的简单转换的标记序列相比，特征序列表现出更多的规律性。在特征级别进行自回归处理，然后使用原始 LLM 的 LM 头导出标记，比直接自回归预测标记产生更好的结果。如图 4 所示，自回归预测特征可产生更好的性能，这通过 1.9 倍的加速比（相比 1.5 倍）更高来证明。

其次，采样过程中固有的不确定性极大地限制了预测下一个特征的性能。在文本生成中，目标 LLM 相应地预测标记和样本的分布，从而引入随机性。特征是高维且连续的，不能以类似的方式处理。如图 3 所示，对“am”或“always”等不同标记进行采样会产生不同的特征序列，从而在特征级自回归中引入歧义。Medusa 在预测间隔标记时面临类似的问题，不确定输入 f_I 的真实目标应该是 p_{am} 还是 p_{always} 。为了解决这个问题，EAGLE 将提前一个时间步的标记序列（包括采样结

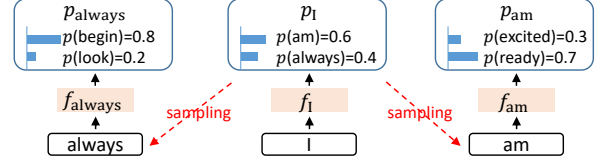


Figure 3: 特征序列的不确定性。 f_I 后面的下一个特征取决于采样结果，不能仅根据 f_I 确定，其中“always”和“am”都可能跟随标记“I”并导致两个分支。

果）输入到草稿模型中。在图 3 所示的示例中，这涉及基于 f_I 和 t_{always} 预测 f_{always} ，以及基于 f_I 和 t_{am} 预测 f_{am} 。如图 4 所示，通过解决不确定性，加速比进一步从 1.9 倍提高到 2.8 倍。

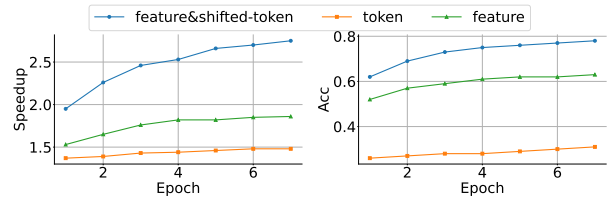


Figure 4: 在温度 = 0 时基于令牌、特征和特征 & 转移令牌的草稿模型的准确性和加速比，在 MT 平台上进行测试，使用 Vicuna 7B 作为原始 LLM。feature&shifted-token 是指使用特征序列和前进一个时间步的 token 序列作为输入。

我们分别使用 MT-bench、HumanEval、GSM8K 和 Alpaca 数据集进行了对话、代码生成、数学推理和指令跟踪任务的实验。测试的法学硕士包括 Vicuna 和 LLaMA2-Chat 系列的所有模型，以及 Mixtral 8x7B Instruct。对于 LLaMA2-Chat 70B，EAGLE 实现了 2.7 倍-3.5 倍的加速比，吞吐量翻倍，理论上保证了生成文本分布的保留。图 1 和 2 说明了 EAGLE 在 MT-bench (Zheng et al., 2023) 上的性能，MT-bench 是一个高度真实的基准，模拟实际应用程序和真实场景，包括类似于与 ChatGPT 对话的多轮指令。我们选择使用这个基准，因为它已被当前最先进的技术（包括 Lookahead 和 Medusa）所采用，以展示它们的加速比。这种选择有助于我们的方法和这些方法之间进行公平和直接的比较。与最近提出的基于推测采样的框架 Lookahead 和 Medusa 相比，EAGLE 分别实现了 1.7 倍-2.1 倍和 1.5 倍-1.6 倍加速。EAGLE 与其他加速或吞吐量提高方法并行运行，例如量化、编译等。将 EAGLE 与这些技术相结合可以进一步降低 LLM 系统的运营成本。例如，借助 gpt-fast (PyTorch Labs, 2023)，EAGLE 在单个 RTX 3090 GPU 上将 LLaMA2-Chat 7B 解码加速至 160.4 个令牌/秒。

EAGLE 拥有较低的训练成本。对于 LLaMA2-Chat 70B 模型，EAGLE 使用 ShareGPT 数据集中不超过 70k 的对话来训练参数少于 1B 的解码器层。训练在 4x A100 (40G) GPU 上于 1-2 天内完成。EAGLE 在

7B、13B 和 33B 模型上的训练甚至可以在 RTX 3090 节点上进行 1-2 天的训练。在实际应用中，EAGLE 只需要一次训练即可为每个查询提供加速。随着查询数量的增加，EAGLE 的摊销训练成本变得可以忽略不计。

除了性能之外，EAGLE 还具有其他优势：

- **概论：** EAGLE 适用于任何自回归法学硕士（至少原则上）。我们已在 MT-bench、GSM8K、HumanEval 和 alpaca 数据集上以零样本方式将 EAGLE 应用于 LLaMA2-Chat (7B、13B、70B)、Vicuna (7B、13B、33B) 和 Mixtral 8x7B Instruct。EAGLE 遵循 LLM 社区中常用的零样本/少样本设置。所有实验均采用相同的权重，仅在 ShareGPT 数据集上进行训练，无需对评估数据集进行任何额外训练。该方法仅向 LLM 添加一个轻量级插件（单个变压器解码器层），可以轻松部署在生产环境中。
- **可靠性：** EAGLE 不涉及对原始 LLM 的任何微调，理论上保证无论是贪婪还是非贪婪设置，EAGLE 都能保留输出分布。这与 Lookahead 和 Medusa 形成鲜明对比，它们要么只关注贪婪设置，要么不保证在这些设置中保留分布。

2. 预赛

注释。 在本文中，“目标 LLM”表示旨在加速的 LLM，而“草案模型”是指用于草案生成的模型。“特征”通常表示 LLM 的第二层特征，即 LM 头之前的隐藏状态。令牌用小写 t 表示，它们的嵌入用 e 表示，特征用 f 表示，分布用 p 表示。序列以大写字母表示，例如， $T_{i:j}$ 代表 $(t_i, t_{i+1}, \dots, t_j)$ 。在 LLM 中，输入 $T_{1:j}$ 通过嵌入层转换为嵌入 $E_{1:j}$ ，然后转换为特征 $F_{1:j}$ ，LM Head 将 f_j 映射到分布 $p_{j+1} = \text{LM_Head}(f_j)$ ，对下一个标记 t_{j+1} 进行采样。对于任何整数 $j \geq 1$ ，令牌级别的普通自回归由 $T_{1:j} \rightarrow E_{1:j} \rightarrow f_j \rightarrow p_{j+1} \rightarrow t_{j+1}$ 描述。

推测性抽样。 推测抽样通过草稿和验证阶段进行，草稿阶段使用较小的模型来生成 γ 代币 $\hat{T}_{j+1:j+\gamma}$ 及其分布 $\hat{P}_{j+1:j+\gamma}$ 。在验证阶段，目标 LLM 的单次前向传递会产生概率 $P_{j+1:j+\gamma}$ 。然后按顺序评估令牌，令牌 $\hat{t}_{j+i}$ 的接受概率为 $\min(1, p_{j+i}(\hat{t}_{j+i})/\hat{p}_{j+i}(\hat{t}_{j+i}))$ 。在拒绝令牌 $\hat{t}_{j+i}$ 后，所有后续令牌都将被丢弃，并且该令牌将根据分布 $\text{norm}(\max(0, p_{j+i} - \hat{p}_{j+i}))$ 重新采样。正如推测性抽样 (Leviathan et al., 2023) 附录 A.1 中所证明的，该方法相当于直接从目标 LLM 抽样。EAGLE 采用这种方法，确保对于贪婪和非贪婪设置，生成文本的分布保持不变。

3. 鹰

EAGLE 与其他基于推测抽样的方法保持一致，包含起草阶段和验证阶段。

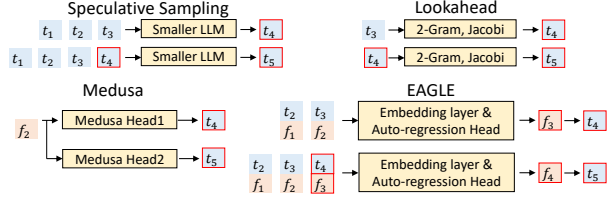


Figure 5: 第四个和第五个令牌 t_4 和 t_5 起草方法的比较。 t (由蓝色块表示) 表示标记， f (橙色块) 表示特征，下标表示它们在序列中的位置。红色边框表示草稿模型的预测。为简单起见，Lookahead 的 n -gram 中的 n (如图所示) 已设置为 2。

3.1. 起草阶段

EAGLE 与其他方法的主要区别主要在于起草阶段。图5说明了不同方法的起草阶段的示意图。推测性采样 (Leviathan et al., 2023; Chen et al., 2023a) 和前瞻 (Fu et al., 2023) 根据标记预测标记。Medusa (Cai et al., 2023) 使用目标 LLM 的特征 f_2 独立预测 t_4 和 t_5 。EAGLE 使用特征序列 (f_1, f_2) 和标记序列 (t_2, t_3) 预测 f_3 ，并提前一个时间步长。从 $p_4 = \text{LM Head}(f_3)$ 开始，对 t_4 进行采样。随后，将 f_3 和 t_4 连接到输入序列中以预测下一个特征 f_4 并对后续标记 t_5 进行采样。

如图6所示，EAGLE 的草稿模型包括三个模块：Embedding 层、LM Head 和 Autoregression Head。Embedding 层和 LM Head 使用目标 LLM 的参数，不需要额外的训练。草稿模型将形状的特征序列 (bs, seq_len, hidden_dim) 和形状的高级标记序列 (bs, seq_len) 作为输入。然后，它将标记序列转换为形状 (bs, seq_len, hidden_dim) 的标记嵌入序列，并将其连接起来形成形状 (bs, seq_len, 2×hidden_dim) 的融合序列。自回归头由 FC 层和解码器层组成。FC 层将融合序列的维度降低到 (bs, seq_len, hidden_dim)，然后我们利用解码器层来预测下一个特征。LM Head 根据特征计算分布，并从中采样下一个标记。最后，将预测特征和采样标记连接到输入中，促进自回归过程的继续。EAGLE 使用树注意力创建树形结构草稿，通过 m 前向传递生成深度为 m 且超过 m 标记的草稿树。例如，如图6所示，EAGLE 仅用 3 次前向传递就起草了一个 10 个令牌树。EAGLE 使用的实际树结构详见附录 A.1。

3.2. 草稿模型的训练

预测下一个特征构成了一个回归任务，为此我们采用 Smooth L1 损失 (见图 5 EAGLE)：

$$L_{reg} = \text{Smooth L1}(f_{i+1}, \text{Draft_Model}(T_{2:i+1}, F_{1:i}))$$

预测特征是草案模型的中间目标，最终目标是预测标记以生成标记序列。因此，我们还利用分类损失来直

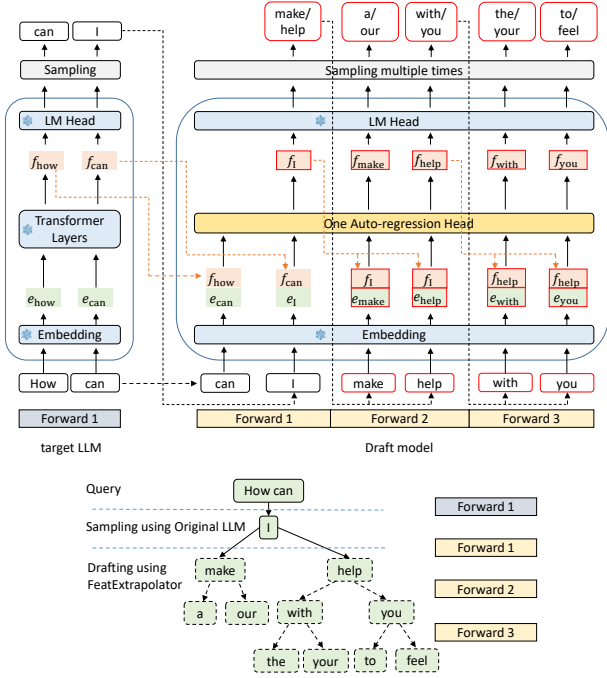


Figure 6: EAGLE 的管道。上半部分说明了计算过程，而下半部分显示了每个步骤相应的生成结果。在上半部分，green块代表 token 嵌入，orange块代表特征，red框代表草稿模型的预测，blue带有雪花图标的模块代表使用目标 LLM 参数，这些参数不受训练。

接优化最终目标：

$$\begin{aligned}
 p_{i+2} &= \text{Softmax}(\text{LM_Head}(f_{i+1})), \\
 \hat{p}_{i+2} &= \text{Softmax}(\text{LM_Head}(\hat{f}_{i+1})), \\
 L_{cls} &= \text{Cross_Entropy}(p_{i+2}, \hat{p}_{i+2})
 \end{aligned}$$

通过整合回归损失和分类损失，我们使用组合损失函数 $L = L_{reg} + w_{cls}L_{cls}$ 训练自回归头。通常，分类损失在数值上比回归损失大一个数量级。因此，我们将 w_{cls} 设置为 0.1。

EAGLE 的自回归头理想地使用目标 LLM 的自回归生成的文本进行训练，但这种方法成本高昂。幸运的是，EAGLE 对训练数据表现出较低的敏感性（第4.3.3节中的消融研究）。我们没有使用目标法学硕士生成的文本，而是使用固定的数据集，从而大大减少了开销。在绘图阶段，EAGLE 自回归处理特征。特征的不准确可能会导致错误累积。为了缓解这个问题，我们通过在训练过程中将从均匀分布 $\mathcal{U}(-0.1, 0.1)$ 采样的随机噪声添加到目标 LLM 的特征（Jain et al., 2023）来采用数据增强。

3.3. 验证阶段

利用树注意力，目标 LLM 通过一次前向传递计算树结构草案中每个标记的概率。在草稿树的每个节点，我们递归地应用推测采样算法来采样或调整分布（详细信息参见附录A.2），与 SpecInfer(Miao et al., 2023) 一致，确保输出文本的分布与目标 LLM 的分布一致。同时，我们记录已接受的令牌及其功能，以便在下一个起草阶段使用。

4. 实验

模型和任务。我们对 Vicuna 模型（7B、13B、33B）、LLaMA2-chat 模型（7B、13B、70B）和 Mixtral 8x7B Instruct 进行了实验，涵盖了当前主流 LLM 的常见尺寸。我们分别使用 MT-bench (Zheng et al., 2023)、HumanEval (Chen et al., 2021)、GSM8K (Cobbe et al., 2021) 和 Alpaca (Taori et al., 2023) 数据集对 EAGLE 进行了多项任务评估，包括多轮对话、代码生成、数学推理和指令跟踪。推测抽样 (Leviathan et al., 2023) 以批量大小 1 进行实验，该设置随后被 DistillSpec (Zhou et al., 2023) 和 BiLD (Kim et al., 2023) 等其他作品采用。同样，我们的大部分实验也采用了这种设置。第4.4节介绍了批量大小大于 1 的实验。

指标。与其他基于推测采样的方法一样，EAGLE 主要关注延迟而不是吞吐量。我们使用以下指标评估加速效果：

- Walltime 加速比：相对于普通自回归解码的实际测试加速比。
- 平均接受长度 τ ：目标 LLM 每次前向传递接受的平均令牌数。
- 接受率 α ：起草过程中接受的令牌与生成的令牌的比率，衡量起草的准确性。它不太适用于树草稿，因为每个位置采样了多个令牌，但只接受了一个。因此，在测量这个指标时，我们利用链草稿而不需要树关注，与推测采样和 DistillSpec 保持一致。EAGLE 的草稿模型输入特征和标记序列。自回归特征处理可能会传播错误，因此我们将接受率测量为 $n-\alpha$ ，考虑到草稿模型预测的 n 特征，可能不准确。

EAGLE 的加速理论上保证了目标 LLM 输出分布的保留。因此，评估 EAGLE 生成结果的质量既没有必要，也没有意义。

训练。我们确定了法学硕士目标。EAGLE 在 ShareGPT 数据集上进行训练，使用 68,000 次对话迭代，学习率设置为 $3e-5$ 。我们采用 AdamW 优化器，将 beta 值 (β_1, β_2) 设置为 (0.9, 0.95)，并实现 0.5 的梯度裁剪。7B、13B、33B、70B 模型对应的 EAGLE 可训练参数分别为 0.24B、0.37B、0.56B、0.99B。MoE 模型 Mixtral 8x7B 的 EAGLE 可训练参数为 0.28B。

Table 1: HumanEval、GSM8K 和 Alpaca 上的加速比和平均接受长度 τ 。T 表示温度，V 表示 Vicuna，LC 表示 LLaMA2-Chat。

		HumanEval		GSM8K		Alpaca	
	Model	Speedup	τ	Speedup	τ	Speedup	τ
T=0	V 7B	3.33x	4.29	3.01x	4.00	2.79x	3.86
	V13B	3.58x	4.39	3.08x	3.97	3.03x	3.95
	V 33B	3.67x	4.28	3.25x	3.94	2.97x	3.61
	LC 7B	3.17x	4.24	2.91x	3.82	2.78x	3.71
	LC 13B	3.76x	4.52	3.20x	4.03	3.01x	3.83
	LC 70B	3.52x	4.42	3.03x	3.93	2.97x	3.77
T=1	V 7B	2.39x	3.43	2.34x	3.29	2.21x	3.30
	V13B	2.65x	3.63	2.57x	3.60	2.45x	3.57
	V 33B	2.76x	3.62	2.77x	3.60	2.52x	3.32
	LC 7B	2.61x	3.79	2.40x	3.52	2.29x	3.33
	LC 13B	2.89x	3.78	2.82x	3.67	2.66x	3.55
	LC 70B	2.92x	3.76	2.74x	3.58	2.65x	3.47

EAGLE 的特点是训练成本低；对于 70B 模型，自回归头可在 1-2 天内 A100 40G 服务器上训练。

4.1. 效力

图1和2以及表1显示了 EAGLE 的加速比。与温度 = 1 相比，EAGLE 在温度 = 0 时表现出更好的加速度。例如，对于 LLaMA2-Chat 13B，在温度 = 0 时，加速比范围从 3.01x 到 3.76x，而在温度 = 1 时，加速比范围从 2.66x 到 2.89x。在代码生成任务 (HumanEval) 中，EAGLE 实现了最佳的加速性能。这归因于代码中固定模板的盛行，使得为这些模板生成草稿变得更容易。与最近推出的基于推测采样的方法 Lookahead 和 Medusa 相比，EAGLE 的速度分别快了 1.70x-2.08x 和 1.47x-1.60x。在 Vicuna 和 LLaMA2-Chat 系列中采用推测采样具有挑战性。对于 7B 模型，没有合适的草案模型。对于其他尺寸，使用 7B 模型作为拔模模型，我们迭代拔模长度从 2 到 10 并报告最高加速比。对于 13B 模型，我们观察到速度没有提高。对于 33B 和 70B 型号，加速比分别为 1.12 倍和 1.88 倍。对于 DistillSpec，为了保证公平性，我们使用了与 EAGLE 相同的训练数据。此外，所采用的散度函数遵循 FKL，如 DistillSpec 论文附录 A.1 中详述。虽然蒸馏略微提高了加速比，但提升幅度有限是因为蒸馏的目的是提高草稿模型的接受率，而投机采样性能的瓶颈在于草稿模型的高开销。

表1和表2表明，在 EAGLE 中，目标 LLM 每次前向传递生成 3.2-4.5 个令牌，超过了每次前向传递仅产生一个令牌的普通解码，从而显著提高了生成速度。如图2和附录B所示，完全准确的特征序列的接受率 $0-\alpha$ 显著超过具有单个错误特征的序列 $1-\alpha$ ，表明特征错误对草稿模型性能的影响。然而， $1-\alpha$ 到 $4-\alpha$ 之间的细微差异强调了 EAGLE 对特征错误的鲁棒性及其对错误累积的熟练处理。

表3 显示 EAGLE 通过 Mixtral 8x7B Instruct 模型实现了 1.5 倍的加速。与 LLaMA 等模型相比，这种适度

Table 2: MT-bench 上的平均接受长度 τ 和接受率 α 。T 表示温度。

	Model	τ	$0-\alpha$	$1-\alpha$	$2-\alpha$	$3-\alpha$	$4-\alpha$
T=0	Vicuna 7B	3.94	0.79	0.74	0.72	0.73	0.67
	Vicuna 13B	3.98	0.79	0.74	0.72	0.74	0.70
	Vicuna 33B	3.68	0.74	0.69	0.67	0.67	0.66
	LLaMA2-Chat 7B	3.62	0.76	0.69	0.67	0.68	0.68
	LLaMA2-Chat 13B	3.90	0.77	0.69	0.69	0.70	0.71
	LLaMA2-Chat 70B	3.81	0.75	0.69	0.65	0.64	0.64
T=1	Vicuna 7B	3.17	0.71	0.68	0.66	0.66	0.65
	Vicuna 13B	3.20	0.73	0.68	0.68	0.67	0.69
	Vicuna 33B	3.22	0.71	0.67	0.64	0.64	0.64
	LLaMA2-Chat 7B	3.30	0.71	0.66	0.66	0.66	0.64
	LLaMA2-Chat 13B	3.45	0.73	0.69	0.66	0.66	0.67
	LLaMA2-Chat 70B	3.46	0.73	0.67	0.64	0.66	0.65

Table 3: 温度 = 0 时 MT 平台上的加速比、平均接受长度 τ 和接受率 α 。目标法学硕士是 Mixtral 8x7B Instruct-v0.1。

Speedup	τ	$0-\alpha$	$1-\alpha$	$2-\alpha$	$3-\alpha$	$4-\alpha$
1.50x	3.25	0.67	0.62	0.61	0.64	0.63

的加速是由于平均接受长度较短以及通过推测采样加速 MoE 模型的复杂性。MoE 模型通常需要在普通自回归解码过程中读取每个令牌仅两名专家的权重。然而，在推测采样的验证阶段，处理多个令牌可能需要访问两个以上专家的权重，这与仅密集解码器模型形成鲜明对比，在密集解码器模型中，无论转发的令牌数量如何，都会读取所有权重。

4.2. 案例研究：EAGLE + gpt-fast

EAGLE 与其他加速技术兼容。我们进行了将 EAGLE 与 gpt-fast 相结合的实验，它采用量化和编译来加速。如图4所示，通过将 EAGLE 与 gpt-fast 集成，我们将单 RTX 3090 上的 LLaMA2-Chat 7B 的生成速度提高到 160.4 个令牌/秒。

Table 4: EAGLE 与 gpt-fast 结合的生成速度，在 MT-bench 上使用 LLaMA2-Chat 7B 在温度 = 0 时进行评估。

Precision	FP16	int4
Vanilla (Huggingface)	24.5 tokens/s	N/A
gpt-fast	55.1 tokens/s	106.9 tokens/s
EAGLE + gpt-fast	100.2 tokens/s	160.4 tokens/s

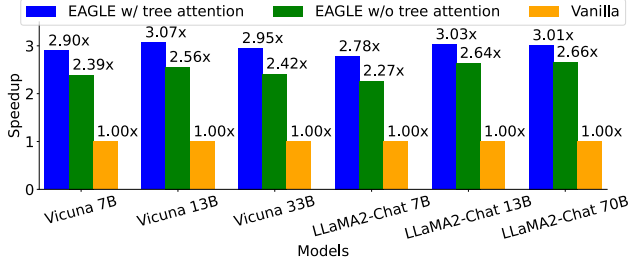


Figure 7: 使用和不使用树注意力的 EAGLE 加速比。评估数据集为 MT-bench，温度参数设置为 0。

Table 5: 使用和不使用树注意力的 EAGLE 的平均接受长度 τ 。评估数据集为 MT-bench，温度参数设置为 0。

Vicuna			LLaMA2-Chat		
Size	Chain	Tree	Size	Chain	Tree
7B	3.20	3.94 (+0.74)	7B	3.00	3.62 (+0.62)
13B	3.23	3.98 (+0.75)	13B	3.18	3.90 (+0.68)
33B	2.97	3.68 (+0.71)	70B	3.12	3.81 (+0.69)

4.3. 消融研究

4.3.1. 树关注

EAGLE 与 SpecInfer 和 Medusa 类似，采用树注意力机制，草稿的生成和验证都是树结构的。相比之下，像推测采样这样的方法不使用树注意力，从而导致链式结构的草稿生成和验证。表5和图7提供了比较结果，表明使用树注意力的影响。在 EAGLE 中实施树草案和验证导致平均接受长度大约增加 0.6-0.8，加速比大约增加 0.3-0.5。与链草稿和验证相比，树草稿和验证不会增加模型（目标 LLM 和草稿模型）中的前向传递数量，但它们确实增加了每个前向传递处理的代币数量。因此，加速比的改善不如平均接受长度的增加明显。值得注意的是，即使不使用树草稿和验证，EAGLE 也表现出显著的加速效果，大约在 2.3x-2.7x 范围内。

4.3.2. 草稿模型的输入

与其他基于推测抽样的方法相比，EAGLE 的关键创新在于它利用了目标 LLM 计算的特征，并将抽样结果纳入草稿模型的输入以解决随机性问题。我们对 Vicuna 7B 进行了消融研究，评估了具有不同输入的草稿模型。我们测试了四种类型的输入：feature&shifted-token (EAGLE)、feature&unshifted-token、token 和 feature。feature&shifted-token (EAGLE) 和 feature&unshifted-token 都集成了不同级别的语义信息。区别在于，特征转移令牌 (EAGLE) 输入提前一步的令牌，使其能够有效地解决随机性问题。除了使用 FC 层来降低特征 & 令牌输入的维度之外，草稿模型的结构保持完全一致。图8展示了以 Vicuna 7B 为目标 LLM 的 MT 平

Table 6: 使用不同训练数据集在 MT 平台上评估的加速比和平均接受长度 τ ，目标 LLM 为 LLaMA2-Chat 7B，温度设置为 0。“固定数据集”指源自 ShareGPT 数据集的问题和答案。“由目标 LLM 生成的数据”表示虽然问题源自 ShareGPT 数据集，但答案由目标 LLM 生成。

Training data	Speedup	τ
Fixed dataset	2.78x	3.62
Data generated by target LLM	2.88x	3.75

台上的实验结果。可以得出三个观察结果。

- 首先，当草稿模型的参数数量有限时，利用特征会产生比令牌稍好的结果。
- 其次，合并特征和令牌会适度提高性能，主要是因为离散、无错误的令牌减轻了特征错误累积，从特征 & 未移动令牌和仅特征草稿模型的类似 $0-\alpha$ 可以明显看出，并且显著改进了 $1-\alpha$ 。
- 第三，解决抽样过程中固有的随机性会带来最显著的改进。与 feature&unshifted-token 相比，feature&shifted-token 方案不会增加复杂性，但通过简单地将令牌推进一个时间步长，显著增强了草稿模型的能力，从而允许草稿模型考虑采样的随机性。

4.3.3. 训练数据

EAGLE 使用固定数据集进行训练，避免使用目标 LLM 生成训练数据而增加开销。消融研究（参见表6）表明，来自目标 LLM 的数据略微提高了性能，表明 EAGLE 对训练数据的敏感性较低，并证明了固定数据集方法降低成本的合理性。

4.4. 批量大小和吞吐量

LLM 中的推理受内存限制 (Patterson, 2004; Shazeer, 2019)，导致 GPU 计算资源未得到充分利用。基于推测采样的方法提高生成速度的原理在于更有效地利用 GPU 计算资源。随着批量大小的增加，GPU 的可用计算能力会减少，导致加速效果降低。在本节中，我们将展示批量大小超过 1 的场景的实验结果。如表7所示，加速比随着批量大小的增加而减小。当使用 Vicuna 7B 作为目标 LLM 时，bs=4 时的加速比高于 bs=3 时的加速比。这是因为，在 EAGLE 的验证阶段，目标 LLM 在一次前向传递中处理多个令牌，并且 bs=4 时的处理速度比 bs=3 时更快。相比之下，在目标 LLM 每次前向传递处理一个令牌的普通自回归解码中，bs=3 和 bs=4 时的速度几乎相同。

尽管基于推测采样的方法主要关注延迟，但我们还研究了 EAGLE 批量大小 > 1 的吞吐量，这是 LLM 系统的

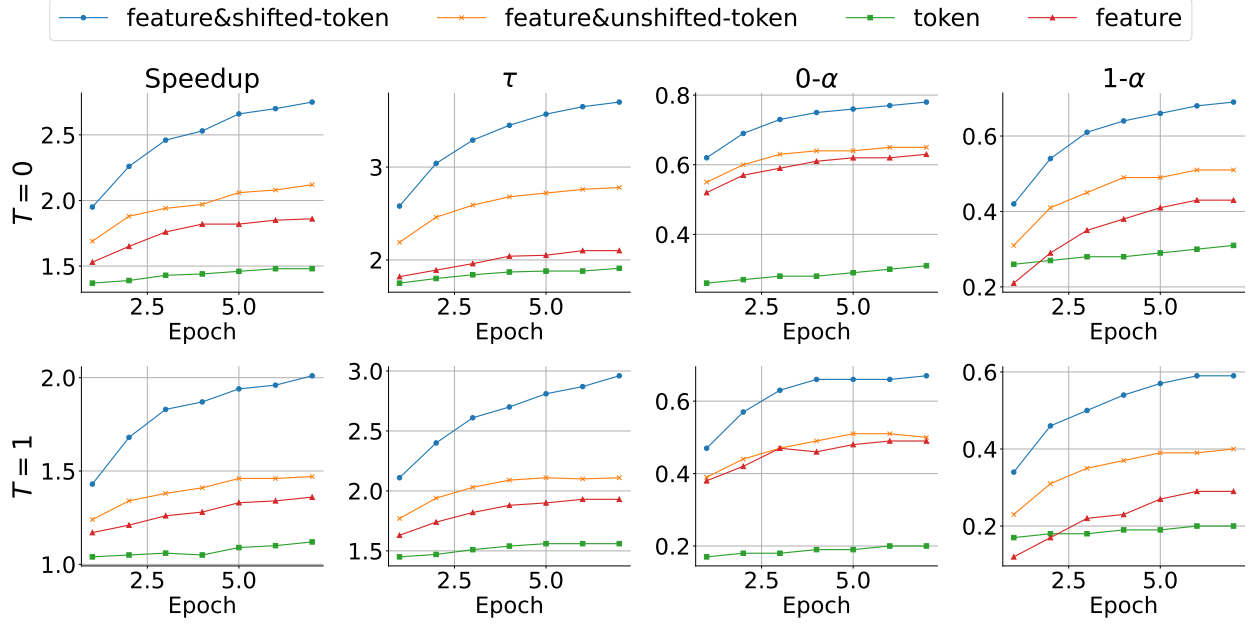


Figure 8: Performance of draft models with varying inputs. The target LLM is Vicuna 7B, and the test dataset is MT-bench. Speed refers to the walltime speedup ratio, τ denotes the average acceptance length, $0-\alpha$ represents the acceptance rate with entirely precise inputs, $1-\alpha$ indicates the acceptance rate when the input includes one imprecise feature, and T refers to the temperature.

Table 7: EAGLE 不同批量大小和吞吐量下的加速比。评估数据集为 MT-bench，温度参数设置为 0。

Batch size	1	2	3	4	Throughput
Vicuna 7B	2.90x	2.87x	2.65x	2.76x	1.97x
LLaMA2-Chat 70B	3.01x	2.81x	2.50x	2.40x	1.99x

另一个关键指标。与普通自回归解码相比，EAGLE 需要稍多的 CUDA 内存。对于作为目标 LLM 的 Vicuna 7B，在具有 24G CUDA 内存的单个 RTX 3090 的内存约束下运行，普通自回归解码和 EAGLE 的最大批量大小 (bs) 分别为 8 和 7。在 LLaMA2-Chat 70B 的情况下，受 4 个 A100 (40G) GPU 总计 160G CUDA 内存的限制，普通自回归解码和 EAGLE 的最大 bs 分别为 5 和 4。所有评估均以 FP16 精度进行。我们计算了不同 bs 的吞吐量并选择了最大值。普通自回归解码和 EAGLE 都在各自的最大 bs 下实现了最大吞吐量。树注意力消耗更多的计算资源。当 bs=7 时，计算资源不太丰富，使得不使用树注意力更加有利。如表7所示，EAGLE 的吞吐量提高了 2 倍。

5. 相关工作

人们对加速语言模型进行了大量研究，涉及蒸馏 (Hinton et al., 2015)、量化 (Hubara et al., 2018; Shen et al., 2020; Kim et al., 2021; Zadeh et al., 2020; Zafrir et al., 2019)、剪枝 (Gale et al., 2019; Sanh et al., 2020; Kur-

tic et al., 2022; Voita et al., 2019) 和创新网络架构设计 (Gu & Dao, 2023; Wu et al., 2020) 等技术。这些方法旨在减少每次前向传递的延迟。

与我们的方法类似的是基于推测抽样的框架。早期的作品 (Stern et al., 2018; Sun et al., 2021) 加速了贪婪解码，而推测采样 (Leviathan et al., 2023; Chen et al., 2023a) 将其扩展到非贪婪采样，可证明保持了原始输出分布。确保输出分布不变使得加速更具挑战性；许多研究都探讨了有损加速作为一种权衡。例如，DistillSpec (Zhou et al., 2023) 使用宽大函数修改接受概率，BiLD (Kim et al., 2023) 如果与目标 LLM 分布的距离度量低于特定阈值则接受草稿，而 Medusa (Cai et al., 2023) 使用硬阈值和熵相关阈值的最小值进行截断。相比之下，EAGLE 不采用任何松弛，并保持 LLM 的输出分布不变。

基于推测抽样的方法之间的主要差异主要体现在起草阶段。推测性抽样 (Leviathan et al., 2023; Chen et al., 2023a) 利用目标 LLM 的较低参数版本作为草稿模型。自推测解码 (Zhang et al., 2023) 在草稿生成期间跳过目标 LLM 的某些层。SpecInfer (Miao et al., 2023) 采用一组小模型来并行生成草稿。级联推测性起草 (Chen et al., 2023b) 和分阶段推测性解码 (Spector & Re, 2023) 级联不同的开销草案模型。在线推测解码 (Liu et al., 2023) 根据查询分布训练草稿模型。Medusa (Cai et al., 2023) 等方法 (Hooper et al., 2023; Fu et al., 2023; Yang et al., 2023b) 不采用单独的目标 LLM；相

反，他们利用目标法学硕士的特征或权重来生成草稿。REST (He et al., 2023) 根据检索方法生成草稿。LLMA (Yang et al., 2023a)，用于输入和输出重叠的语法纠正等任务，直接从输入中检索草稿。

6. 结论

在本文中，我们介绍了 EAGLE，一种高效的推测采样框架。EAGLE 在更结构化（第二层到顶层）特征级别上自回归地执行起草过程，并通过合并前面一个时间步的标记来减轻预测下一个特征时的采样不确定性。EAGLE 保证保留 LLM 的输出分布，同时显著提高生成速度。在 MT-bench 上，EAGLE 比普通自回归解码快 2.1 倍到 3.8 倍，比 Lookahead 快 1.7 倍到 2.1 倍，比 Medusa 快 1.5 倍到 1.6 倍。

致谢。我们感谢与 Medusa 团队负责人 Tianle Cai、Lookahead 团队负责人张浩、SpecTr 团队负责人 Ziteng Sun、与 X 上 gpt-fast 团队负责人 Horace He 和 Soumith Chintala 以及 Yihan Wu 的互动。

References

- Cai, T., Li, Y., Geng, Z., Peng, H., and Dao, T. Medusa: Simple framework for accelerating LLM generation with multiple decoding heads. <https://github.com/FasterDecoding/Medusa>, 2023.
- Chen, C., Borgeaud, S., Irving, G., Lepiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023a.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.
- Chen, Z., Yang, X., Lin, J., Sun, C., Huang, J., and Chang, K. C.-C. Cascade speculative drafting for even faster LLM inference. arXiv preprint arXiv:2312.11462, 2023b.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Breaking the sequential dependency of LLM inference using lookahead decoding, November 2023. URL <https://lmsys.org/blog/2023-11-21-lookahead-decoding/>.
- Gale, T., Elsen, E., and Hooker, S. The state of sparsity in deep neural networks.(2019). arXiv preprint cs.LG/1902.09574, 2019.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.
- He, Z., Zhong, Z., Cai, T., Lee, J. D., and He, D. Rest: Retrieval-based speculative decoding. arXiv preprint arXiv:2311.08252, 2023.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.
- Hooper, C., Kim, S., Mohammadzadeh, H., Genc, H., Keutzer, K., Gholami, A., and Shao, S. Speed: Speculative pipelined execution for efficient decoding. arXiv preprint arXiv:2310.12072, 2023.
- Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., and Bengio, Y. Quantized neural networks: Training neural networks with low precision weights and activations. journal of machine learning research, 18 (187):1–30, 2018.
- Jain, N., Chiang, P.-y., Wen, Y., Kirchenbauer, J., Chu, H.-M., Somepalli, G., Bartoldson, B. R., Kailkhura, B., Schwarzschild, A., Saha, A., et al. NEFTune: Noisy embeddings improve instruction finetuning. arXiv preprint arXiv:2310.05914, 2023.
- Kim, S., Gholami, A., Yao, Z., Mahoney, M. W., and Keutzer, K. I-bert: Integer-only bert quantization. In International conference on machine learning, pp. 5506–5518. PMLR, 2021.
- Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. Speculative decoding with big little decoder. In Thirty-seventh Conference on Neural Information Processing Systems, 2023.
- Kurtic, E., Campos, D., Nguyen, T., Frantar, E., Kurtz, M., Fineran, B., Goin, M., and Alistarh, D. The optimal bert surgeon: Scalable and accurate second-order pruning for large language models. arXiv preprint arXiv:2203.07259, 2022.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning, pp. 19274–19286. PMLR, 2023.
- Liu, X., Hu, L., Bailis, P., Stoica, I., Deng, Z., Cheung, A., and Zhang, H. Online speculative decoding. arXiv preprint arXiv:2310.07177, 2023.

- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Wong, R. Y. Y., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. SpecInfer: Accelerating generative LLM serving with speculative inference and token tree verification. arXiv preprint arXiv:2305.09781, 2023.
- Patterson, D. A. Latency lags bandwidth. *Communications of the ACM*, 47(10):71–75, 2004.
- PyTorch Labs. gpt-fast. <https://github.com/pytorch-labs/gpt-fast/>, 2023.
- Sanh, V., Wolf, T., and Rush, A. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, 33:20378–20389, 2020.
- Santilli, A., Severino, S., Postolache, E., Maiorca, V., Mancusi, M., Marin, R., and Rodola, E. Accelerating transformer inference for translation via parallel decoding. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12336–12355, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.689. URL <https://aclanthology.org/2023.acl-long.689>.
- Shazeer, N. Fast transformer decoding: One write-head is all you need. arXiv preprint arXiv:1911.02150, 2019.
- Shen, S., Dong, Z., Ye, J., Ma, L., Yao, Z., Gholami, A., Mahoney, M. W., and Keutzer, K. Q-bert: Hessian based ultra low precision quantization of bert. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8815–8821, 2020.
- Spector, B. and Re, C. Accelerating LLM inference with staged speculative decoding. arXiv preprint arXiv:2308.04623, 2023.
- Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.
- Sun, X., Ge, T., Wei, F., and Wang, H. Instantaneous grammatical error correction with shallow aggressive decoding. arXiv preprint arXiv:2106.04970, 2021.
- Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. LLaMA 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.
- Voita, E., Talbot, D., Moiseev, F., Sennrich, R., and Titov, I. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. arXiv preprint arXiv:1905.09418, 2019.
- Wu, Z., Liu, Z., Lin, J., Lin, Y., and Han, S. Lite transformer with long-short range attention. arXiv preprint arXiv:2004.11886, 2020.
- Xia, H., Ge, T., Wang, P., Chen, S.-Q., Wei, F., and Sui, Z. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3909–3925, 2023.
- Yang, N., Ge, T., Wang, L., Jiao, B., Jiang, D., Yang, L., Majumder, R., and Wei, F. Inference with reference: Lossless acceleration of large language models. arXiv preprint arXiv:2304.04487, 2023a.
- Yang, S., Lee, G., Cho, J., Papailiopoulos, D., and Lee, K. Predictive pipelined decoding: A compute-latency trade-off for exact llm decoding. arXiv preprint arXiv:2307.05908, 2023b.
- Zadeh, A. H., Edo, I., Awad, O. M., and Moshovos, A. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 811–824. IEEE, 2020.
- Zafrir, O., Boudoukh, G., Izsak, P., and Wasserblat, M. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pp. 36–39. IEEE, 2019.
- Zhang, J., Wang, J., Li, H., Shou, L., Chen, K., Chen, G., and Mehrotra, S. Draft & verify: Lossless large language model acceleration via self-speculative decoding. arXiv preprint arXiv:2309.08168, 2023.
- Zhang, P., Zeng, G., Wang, T., and Lu, W. TinyLlama: An open-source small language model. arXiv preprint arXiv:2401.02385, 2024.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al. Judging llm-as-a-judge with mt-bench and chatbot arena. arXiv preprint arXiv:2306.05685, 2023.

Zhou, Y., Lyu, K., Rawat, A. S., Menon, A. K., Rostamizadeh, A., Kumar, S., Kagy, J.-F., and Agarwal, R. DistillSpec: Improving speculative decoding via knowledge distillation. arXiv preprint arXiv:2310.08461, 2023.

A. 实施细节

A.1. 树结构

利用树注意力，EAGLE 生成树结构草案。图9的左侧说明了草稿的树结构，而右侧则描述不使用树注意力时相应的链结构草稿（如第4.3.1节中详细介绍的消融研究中所使用的）。在贪婪设置中，我们选择概率最高的前 k 标记作为子节点。在非贪婪设置中，我们对 k 令牌进行采样。子节点的数量 k 可以从图 9 中推断出来；例如，根节点处的 $k = 4$ 。无论采用树形结构还是链式结构草图，草图模型在草图阶段都会经历 5 次前向传递。在验证阶段，每个令牌的概率是通过目标 LLM 的单次前向传递获得的。

为什么我们要用这样的树结构呢？如图 9 所示，树结构的选择并未经过严格优化，而是基于直觉：较高概率标记的分支应该更深、更宽。对于本文，所有实验中的所有模型都使用图 9 中所示的草稿结构。但是，最佳树结构可能与上下文相关。例如，随着批量大小的增加和冗余计算资源的减少，较小的树可能会更好。调整草案结构可能会提高性能。

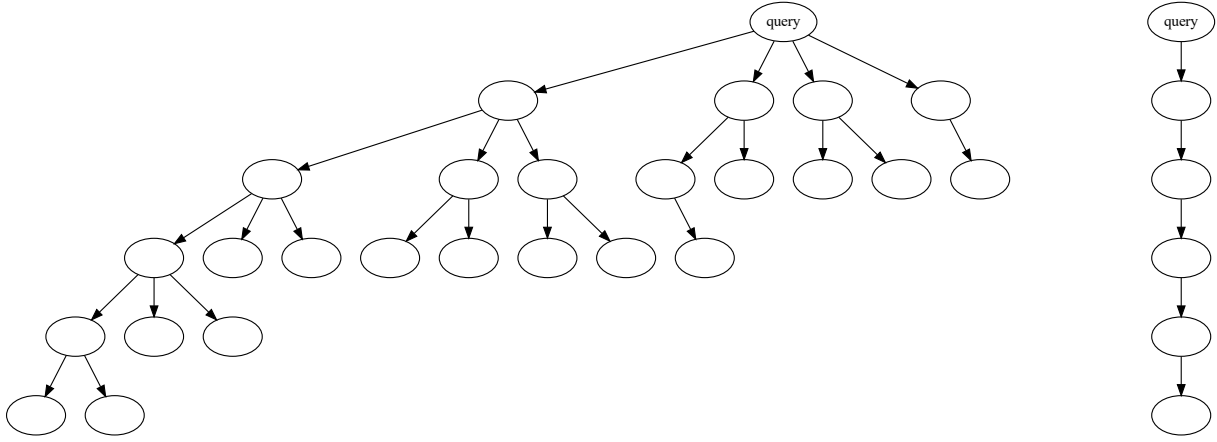


Figure 9: EAGLE 草案的结构。左侧显示了使用树注意力时的草稿结构，而右侧则描绘了不使用树注意力的草稿结构。

A.2. 多轮推测采样

与推测采样的链式结构草案不同，EAGLE 采用树形结构草案，需要对采样算法进行修改。推测采样的采样算法 A 可以简单描述为：如果接受了一个 token，则返回该 token；如果拒绝了，则返回该 token；如果拒绝了，则返回该 token。否则，它会从调整后的分布中采样一个令牌。对于具有 k 候选令牌的树形结构草案，多轮推测采样递归调用算法 A。多轮推测采样不是在拒绝令牌后直接从调整后的分布中采样，而是再次调用 A。如果所有令牌都被拒绝，则直接从调整后的分布中采样。算法1中提供了多轮推测采样的伪代码。

B. 详细实验结果

表8显示了 EAGLE 在 HumanEval、GSM8K 和 Alpaca 数据集上的加速比、平均接受长度 τ 和接受率 α 。

Algorithm 1 多轮推测抽样

输入： 目标分布 p 、样本 t_i 和分布 $\hat{p}_i$ ，每个 i 从 1 到 k ，其中 t_i 是从 $\hat{p}_i$ 采样的，
输出： 样本 $x \sim p$;
 $i \leftarrow 1$
 for $i \leq k$ do
 $r \leftarrow U(0, 1)$
 if $r < p(t_i)/\hat{p}_i(t_i)$ then
 返回 t_i
 end if
 $p \leftarrow \text{norm}(\max(0, p(t_i) - \hat{p}_i(t_i)))$
 $i \leftarrow i + 1$
 end for
 样品 $t \sim p$
返回 t

Table 8: HumanEval、GSM8K 和 Alpaca 在温度 = 0 时的加速比、平均接受长度 τ 和接受率 α 。

Dataset	Model	Speedup	τ	0- α	1- α	2- α	3- α	4- α
HumanEval	Vicuna 7B	3.33x	4.29	0.82	0.77	0.72	0.69	0.71
	Vicuna13B	3.58x	4.39	0.85	0.78	0.74	0.72	0.73
	Vicuna 33B	3.67x	4.28	0.83	0.77	0.74	0.70	0.70
	LLaMA2-Chat 7B	3.17x	4.24	0.81	0.76	0.73	0.74	0.72
	LLaMA2-Chat 13B	3.76x	4.52	0.85	0.80	0.78	0.76	0.75
	LLaMA2-Chat 70B	3.52x	4.42	0.84	0.79	0.75	0.73	0.74
GSM8K	Vicuna 7B	3.01x	4.00	0.79	0.71	0.70	0.71	0.70
	Vicuna13B	3.08x	3.97	0.79	0.71	0.67	0.68	0.64
	Vicuna 33B	3.25x	3.94	0.79	0.71	0.67	0.67	0.67
	LLaMA2-Chat 7B	2.91x	3.82	0.75	0.69	0.64	0.65	0.63
	LLaMA2-Chat 13B	3.20x	4.03	0.80	0.70	0.70	0.68	0.66
	LLaMA2-Chat 70B	3.03x	3.93	0.77	0.71	0.66	0.64	0.60
Alpaca	Vicuna 7B	2.79x	3.86	0.74	0.68	0.66	0.66	0.67
	Vicuna13B	3.03x	3.95	0.72	0.67	0.64	0.63	0.64
	Vicuna 33B	2.97x	3.61	0.70	0.64	0.64	0.63	0.64
	LLaMA2-Chat 7B	2.78x	3.71	0.73	0.66	0.62	0.64	0.62
	LLaMA2-Chat 13B	3.01x	3.83	0.75	0.67	0.64	0.63	0.63
	LLaMA2-Chat 70B	2.97x	3.77	0.76	0.68	0.65	0.61	0.62